

Platform Amendment — Version 4.6.0

Vermont Health Platform (HTR)

Type: Amendment to Platform Documentation (supplements v4.5.0, v4.4.0, v4.3.0, and printed v4.2.0 docs)
Version: 4.6.0 **Date:** April 2026 **Classification:** Internal **Scope:** Two new platform features: State Peer Comparison (confirmed complete and documented), and Wire Article Discussion Threads — full-stack implementation including database, API, real-time subscriptions, and frontend drawer UI.

Table of Contents

- 1. [Summary of Changes](#)
- 2. [State Peer Comparison — What Exists and How It Works](#)
- 3. [Wire Discussion Threads — Feature Overview](#)
- 4. [Database — Migration 025](#)
- 5. [API Layer — Four Routes](#)
- 6. [WireCommentDrawer Component](#)
- 7. [WireFeed Updates](#)
- 8. [Deployment Checklist](#)
- 9. [Architecture Reference Diagram](#)
- 10. [Known Issues Register](#)

1. Summary of Changes

#	Area	Change	Files Affected
1	Frontend	State Peer Comparison — audited and confirmed complete	app/dashboard/compare/page.tsx , app/dashboard/compare/CompareClient.tsx
2	Database	Migration 025 — wire_comments and wire_comment_upvotes tables, trigger, view, RLS, Realtime	supabase/migrations/025_wire_comments.sql
3	API	GET /api/wire/comments — load comments for one article	app/api/wire/comments/route.ts
4	API	POST /api/wire/comments — create a comment (auth required)	app/api/wire/comments/route.ts
5	API	DELETE /api/wire/comments/[id] — delete own comment (auth required)	app/api/wire/comments/[id]/route.ts
6	API	GET /api/wire/comments/counts — all article comment counts for badges	app/api/wire/comments/counts/route.ts
7	Frontend	WireCommentDrawer — slide-in drawer with live Realtime updates	components/WireCommentDrawer.tsx
8	Frontend	WireFeed — Discuss button and count badge per article	app/the-wire/WireFeed.tsx
9	Frontend	TheWirePage server component — passes currentUserId to feed	app/the-wire/page.tsx

2. State Peer Comparison — What Exists and How It Works

Audit Finding

When planning the State Comparison feature, a code audit revealed it was already fully implemented from an earlier development session. Rather than rebuild it, this amendment documents how it works so the team understands what has already been shipped.

Where to Find It

- URL: `/dashboard/compare`
- Linked from: the Dashboard index page (`/dashboard`) via a "Compare states →" button in the top-right corner
- Page entry: `frontend/app/dashboard/compare/page.tsx` (server component)
- Interactive UI: `frontend/app/dashboard/compare/CompareClient.tsx` (client component)

What It Does

The comparison tool lets a user select up to 4 US states simultaneously and displays them side-by-side in a structured table, comparing every dimension in the Health Transformation Readiness (HTR) performance index.

Metrics compared:

Metric	Key path in data	Lower-is-better?
Overall HTR Score	<code>performanceScore</code>	No
VBP Adoption	<code>metrics.policy.vbpAdoption</code>	No
Telehealth Policy	<code>metrics.policy.telehealth</code>	No
Spending per Capita	<code>metrics.economics.spendingPerCapita</code>	Yes
Insurance Coverage	<code>metrics.economics.insuranceCoverage</code>	No
EHR Adoption	<code>metrics.technology.ehrAdoption</code>	No
Broadband Access	<code>metrics.technology.broadbandAccess</code>	No
Preventive Care	<code>metrics.clinical.preventiveCare</code>	No
Readmission Rate	<code>metrics.clinical.readmissionRate</code>	Yes
Racial Equity Gap	<code>metrics.equity.racialEquityGap</code>	No
SDOH Integration	<code>metrics.equity.sdoHIntegration</code>	No

The tool is aware of which metrics are "lower is better" — for Spending per Capita and Readmission Rate, the state with the **lowest** value gets the "best" highlight, not the highest.

Key UI Behaviors

State selector (left sidebar):

- Lists all states alphabetically
- Filter box to find a state by name quickly
- Clicking a state toggles it in/out of the comparison
- Once 4 states are selected, all unselected states are disabled (greyed out) to enforce the 4-state maximum
- Checkbox to toggle "Highlight best value" — when on, the best cell in each row gets a green background and a tiny "best" label

Comparison table (main area):

- First column: metric label
- Remaining columns: one per selected state
- State name in column header links to that state's full profile page
- Status badge under each state name (Leading / Improving / Stable / At Risk)
- Bottom row: narrative summary excerpt from each state's profile, with a "Full profile →" link

Data source: The comparison page calls `getAllPerformanceIndexes()` — a Sanity CMS query that fetches all state profiles in one request. The page has `revalidate = 3600` (Sanity data is re-fetched at most once per hour).

How `getNestedValue` Works

The metric keys use dot notation (e.g. `"metrics.policy.vbpAdoption"`) to reach deeply nested properties in the state profile object. The helper function walks the object by splitting on `.`:

```
function getNestedValue(obj: PerformanceIndexProfile, path: string): number {
  return path.split(".").reduce((acc: unknown, key) => {
    if (acc && typeof acc === "object") return (acc as Record<string, unknown>)[key];
    return undefined;
  }, obj) as number;
}
```

This means adding a new metric to the comparison table requires only adding one entry to the `METRICS` array — no other code changes needed:

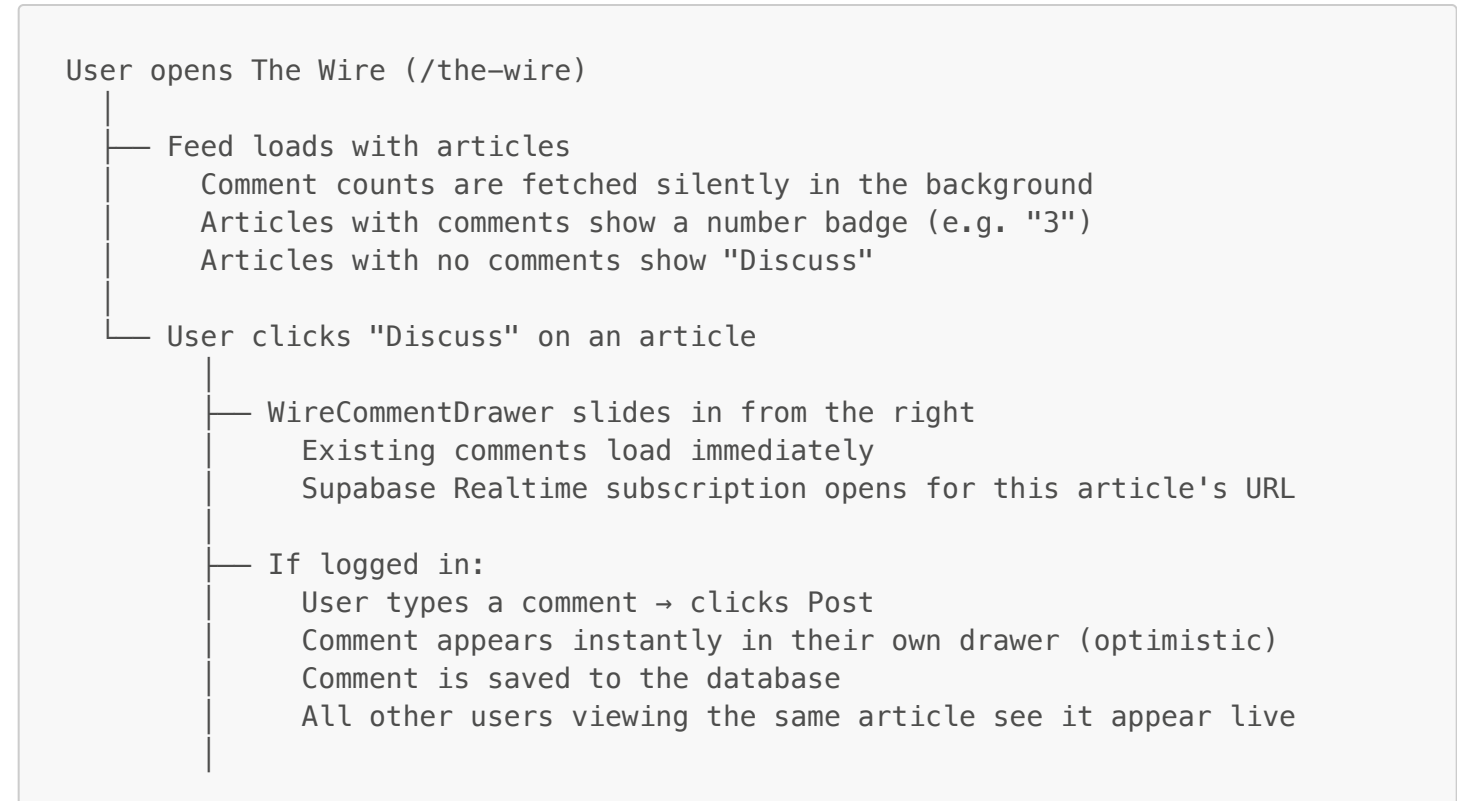
```
const METRICS = [
  { key: "metrics.clinical.readmissionRate", label: "Readmission Rate", format: (v: number) => `${v ?? "-"}%` },
  // Adding a new metric is just this:
  { key: "metrics.clinical.somethingNew", label: "My New Metric", format: (v: number) => `${v ?? "-"}%` },
];
```

3. Wire Discussion Threads — Feature Overview

What Was Built

Every article in The Wire feed now has a discussion thread. A "Discuss" button appears beneath each article headline. Clicking it slides open a panel on the right side of the screen showing all existing comments for that specific article, a text input to post a new comment, and live updates — if another user posts while the drawer is open, their comment appears automatically without any page refresh.

User Experience Flow



└─ If not logged in:
"Sign in to join the discussion" prompt with login link

Design Decisions Explained

Articles are identified by URL, not a database ID. Wire articles are RSS feed items — they have no ID in our database. Rather than creating a separate `wire_articles` table and syncing it with the RSS feed (complex, fragile), comments are linked to articles by their canonical URL (the `<link>` element from the RSS feed). This is simple and robust: the same URL always refers to the same article regardless of when the feed is fetched.

No article body is stored, only the title. When a comment is posted, the article title is stored alongside the URL in the comment row. This allows the drawer header to show the article title even if the article later disappears from the live feed.

Realtime updates via Supabase, not polling. Rather than having the client check for new comments every few seconds (polling), the drawer subscribes to Supabase Realtime — PostgreSQL's logical replication stream. When any user inserts a comment on a given article, all other users viewing that same article's drawer receive the event instantly via a persistent WebSocket connection.

Comments are public-readable, write-protected. Anonymous visitors can read all comments. Only authenticated users can post. Authors can delete their own comments. Admins cannot delete comments through the regular UI (they can do so through Supabase directly if needed).

4. Database — Migration 025

File: `supabase/migrations/025_wire_comments.sql`

How to Apply This Migration

This migration must be run **once** in your Supabase project. There are two ways to do this:

Option A — Supabase Dashboard (recommended for non-developers):

1. Log in to supabase.com and open your project
2. Click **SQL Editor** in the left sidebar
3. Click **New query**
4. Open the file `supabase/migrations/025_wire_comments.sql` in a text editor, copy all of its contents
5. Paste into the SQL Editor
6. Click **Run**
7. You should see "Success. No rows returned."

Option B — Supabase CLI (for developers):

```
supabase db push
```

The CLI reads all migration files in `supabase/migrations/` in order and applies any that haven't been applied yet.

What the Migration Creates

Table: `wire_comments`

```
CREATE TABLE public.wire_comments (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  article_url TEXT          NOT NULL,  
  article_title TEXT        NOT NULL DEFAULT '',  
  user_id     UUID          NOT NULL REFERENCES auth.users(id) ON DELETE  
  CASCADE,
```

```
body          TEXT          NOT NULL CHECK (char_length(body) BETWEEN 2 AND 2000),
upvotes       INT           NOT NULL DEFAULT 0,
created_at    TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Column explanations:

- `id` — UUID primary key, auto-generated. Every comment gets a unique identifier.
- `article_url` — The full URL of the RSS article (e.g. `https://kffhealthnews.org/news/article/...`). This is the join key between comments and the article they belong to.
- `article_title` — The article title at time of posting. Stored for display in the drawer header even if the article ages out of the feed.
- `user_id` — Foreign key to `auth.users`. `ON DELETE CASCADE` means if a user's account is deleted, all their comments are automatically deleted too.
- `body` — The comment text. The `CHECK` constraint enforces a minimum of 2 characters (prevents empty/whitespace-only posts) and a maximum of 2000 characters. This constraint is enforced at the database level regardless of what the API or UI does.
- `upvotes` — A denormalized counter kept in sync by a database trigger (see below). Storing the count directly in the row avoids a `COUNT(*)` query every time comments are loaded.
- `created_at` — Timestamp of posting. Displayed in the drawer as a relative time string ("3h ago", "2d ago").

Indexes:

```
CREATE INDEX wire_comments_article_idx ON wire_comments(article_url);
CREATE INDEX wire_comments_user_idx   ON wire_comments(user_id);
CREATE INDEX wire_comments_created_idx ON wire_comments(created_at DESC);
```

- `article_idx` — Used by every query that loads comments for one article. Without this, Supabase would scan the entire table.
- `user_idx` — Used when deleting a user's comments or checking ownership.
- `created_idx` — Used for ordering comments chronologically. `DESC` index makes "most recent first" queries faster (not currently used, but useful if sort order changes).

Table: `wire_comment_upvotes`

```
CREATE TABLE public.wire_comment_upvotes (
  user_id      UUID NOT NULL REFERENCES auth.users(id)      ON DELETE CASCADE,
  comment_id   UUID NOT NULL REFERENCES wire_comments(id)   ON DELETE CASCADE,
  PRIMARY KEY (user_id, comment_id)
);
```

This is a **junction table** — it records which user has upvoted which comment. The composite primary key (`user_id, comment_id`) makes it impossible for the same user to upvote the same comment twice at the database level. Both foreign keys have `ON DELETE CASCADE`:

- If the user is deleted → their upvotes are deleted
- If the comment is deleted → upvotes on that comment are deleted

Note: The upvote UI is scaffolded in the database but the frontend upvote button is not yet implemented. The table and trigger are ready; wiring the button is a future step.

Trigger: `sync_wire_comment_upvotes`

```
CREATE OR REPLACE FUNCTION sync_wire_comment_upvotes()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE wire_comments SET upvotes = upvotes + 1 WHERE id =
NEW.comment_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE wire_comments SET upvotes = upvotes - 1 WHERE id =
OLD.comment_id;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_wire_upvote
AFTER INSERT OR DELETE ON wire_comment_upvotes
FOR EACH ROW EXECUTE FUNCTION sync_wire_comment_upvotes();
```

How triggers work in Postgres: A trigger is a function that runs automatically when a specific event happens on a table. Here, every time a row is inserted into `wire_comment_upvotes` (user upvotes a comment), the trigger fires and increments the `upvotes` counter on the corresponding `wire_comments` row. Every time a row is deleted (user removes their upvote), it decrements. This keeps the counter perfectly in sync without requiring any application code to manage it.

`RETURN NULL` is used (rather than `RETURN NEW` or `RETURN OLD`) because this is an `AFTER` trigger on a different table — it has no row to return.

View: `wire_comment_counts`

```
CREATE OR REPLACE VIEW wire_comment_counts AS
SELECT article_url, COUNT(*) AS comment_count
FROM wire_comments
GROUP BY article_url;
```

This view pre-aggregates comment counts per article URL. The `/api/wire/comments/counts` route reads from this view to populate the comment badges in the feed. Using a view rather than a raw `GROUP BY` query in the API keeps the SQL out of application code and makes it easy to change the aggregation logic later (e.g. add a `WHERE deleted_at IS NULL` filter if soft-delete is added).

Row Level Security (RLS)

RLS is Supabase's mechanism for controlling which rows a user can read, insert, update, or delete. It is enforced at the database level — it cannot be bypassed by the application layer (though the service-role key bypasses it, which is why it's only used server-side).

```
-- Public read: anyone (including logged-out users) can read comments
CREATE POLICY "wire_comments_public_read"
ON wire_comments FOR SELECT USING (true);

-- Insert: must be logged in, and the user_id must match the logged-in user
CREATE POLICY "wire_comments_insert"
ON wire_comments FOR INSERT
WITH CHECK (auth.uid() IS NOT NULL AND auth.uid() = user_id);

-- Delete: only the author of the comment
CREATE POLICY "wire_comments_delete"
ON wire_comments FOR DELETE
USING (auth.uid() = user_id);
```

```
-- Upvotes: users can only touch their own upvote rows
CREATE POLICY "wire_upvotes_own"
  ON wire_comment_upvotes FOR ALL
  USING (user_id = auth.uid());
```

Why both `auth.uid()` IS NOT NULL and `auth.uid() = user_id` on insert? The first check ensures the user is logged in at all. The second ensures the `user_id` they're trying to write matches their actual identity — without this, a logged-in user could forge a comment under another user's ID by sending a crafted API request.

Enabling Realtime

```
ALTER PUBLICATION supabase_realtime ADD TABLE wire_comments;
```

What this does: Supabase Realtime works by subscribing to PostgreSQL's logical replication log — a stream of all INSERT/UPDATE/DELETE events on a table. The `supabase_realtime` publication controls which tables are included in that stream. This line adds `wire_comments` to the publication, which is what allows the `WireCommentDrawer` component to receive live events when new comments are posted.

Important: This SQL line is safe to run even if Realtime is not yet enabled in your Supabase project settings. If Realtime is off, the client subscription simply won't receive any events — it will not throw an error. Turn on Realtime for the `wire_comments` table from: **Supabase Dashboard → Database → Replication → `wire_comments` → toggle on.**

5. API Layer — Four Routes

The Wire discussion feature is served by four Next.js API route handlers under `frontend/app/api/wire/comments/`.

Route 1: GET `/api/wire/comments`

File: `frontend/app/api/wire/comments/route.ts`

Purpose: Fetch all comments for a single article, including the author's display name.

Request:

```
GET /api/wire/comments?url=https://kffhealthnews.org/news/article/example
```

Response (200):

```
{
  "comments": [
    {
      "id": "uuid",
      "body": "Great analysis of the Medicare impact here.",
      "upvotes": 0,
      "created_at": "2026-04-03T14:22:00Z",
      "user_id": "uuid",
      "profiles": {
        "full_name": "Jane Smith",
        "avatar_url": null
      }
    }
  ]
}
```


Who can call this: Anyone — no authentication required. Anonymous users can read comments.

Key implementation detail — the profile join:

```
const { data } = await dbAdmin
  .from("wire_comments")
  .select(`
    id, body, upvotes, created_at, user_id,
    profiles!wire_comments_user_id_fkey ( full_name, avatar_url )
  `)
  .eq("article_url", url)
  .order("created_at", { ascending: true });
```

The `profiles!wire_comments_user_id_fkey` syntax is Supabase's way of specifying a foreign key join. The `!wire_comments_user_id_fkey` suffix tells Supabase which foreign key constraint to use for the join (because there could be multiple FK relationships between two tables). This fetches the author's name in a single query rather than requiring a separate lookup per comment.

Why `dbAdmin` (service role) for a public read? The `profiles` table has RLS policies that restrict who can see which profile rows. Since this is a server-side API route (not running in the browser), using the service-role client means the join will always succeed regardless of the requesting user's role. The calling user cannot access the service-role key — it is never sent to the browser.

Route 2: POST `/api/wire/comments`

File: `frontend/app/api/wire/comments/route.ts` (same file, different HTTP method)

Purpose: Post a new comment on an article.

Authentication: Required. Unauthenticated requests are rejected by `requireAuth()`, which reads the session cookie and throws a redirect to `/login` if not logged in.

Request body:

```
{
  "article_url": "https://kffhealthnews.org/news/article/example",
  "article_title": "Senate Passes Medicare Funding Bill",
  "body": "This is the first major change to Medicare in a decade."
}
```

Response (201 on success):

```
{
  "comment": {
    "id": "new-uuid",
    "body": "This is the first major change to Medicare in a decade.",
    "upvotes": 0,
    "created_at": "2026-04-03T14:25:00Z",
    "user_id": "user-uuid",
    "profiles": { "full_name": "John Doe", "avatar_url": null }
  }
}
```

Validation:

- `article_url` must be present
- `body` must be present, between 2 and 2000 characters (enforced at API layer; the database `CHECK` constraint also enforces this as a second line of defense)

- The inserting user's `user_id` is taken from the session — the client cannot specify it

Why the full comment (with profile) is returned on create: The `WireCommentDrawer` adds the returned comment directly to its local state list (optimistic UI). To display it correctly, it needs the `profiles` join already resolved. Returning the full row with the join avoids a separate fetch after posting.

Route 3: DELETE `/api/wire/comments/[id]`

File: `frontend/app/api/wire/comments/[id]/route.ts`

Purpose: Delete a specific comment. Only the comment's author can do this.

Authentication: Required.

Request:

```
DELETE /api/wire/comments/a1b2c3d4-...
```

Response (200 on success):

```
{ "ok": true }
```

Error responses:

- `404` — comment ID not found
- `403` — comment exists but belongs to a different user
- `500` — database error

Why ownership is checked in application code even though RLS policies exist?

The route uses `dbAdmin` (service role), which bypasses RLS. This is intentional — the route needs to first fetch the comment's `user_id` to compare it against the session user. If the anon client were used instead, RLS would block the read itself (since non-authors can't read `user_id` directly via RLS delete policies). The application code therefore does the authorization check explicitly:

```
const { data: comment } = await dbAdmin
  .from("wire_comments")
  .select("user_id")
  .eq("id", id)
  .single();

if (!comment) return NextResponse.json({ error: "not found" }, { status: 404 });
if (comment.user_id !== user.id) {
  return NextResponse.json({ error: "forbidden" }, { status: 403 });
}
```

This is the correct pattern when using the service-role client for operations that have authorization implications — the application code owns the authorization logic rather than relying on RLS.

Route 4: GET `/api/wire/comments/counts`

File: `frontend/app/api/wire/comments/counts/route.ts`

Purpose: Return a flat map of `{ article_url → comment_count }` for all articles that have at least one comment. Used to populate the comment count badges in the Wire feed without making one request per article.

Request:

```
GET /api/wire/comments/counts
```

Response (200):

```
{
  "counts": {
    "https://kffhealthnews.org/news/article/example": 3,
    "https://www.statnews.com/2026/04/01/example": 1
  }
}
```

Articles with zero comments are not included — a missing key in the `counts` object means zero. The feed uses `commentCounts[item.url] ?? 0` to handle this.

This route reads from the `wire_comment_counts` database view, which does the `GROUP BY article_url` aggregation. On failure (e.g. database error), it returns `{ counts: {} }` — an empty object, which just means all articles show "Discuss" instead of a number. The feed degrades gracefully.

6. WireCommentDrawer Component

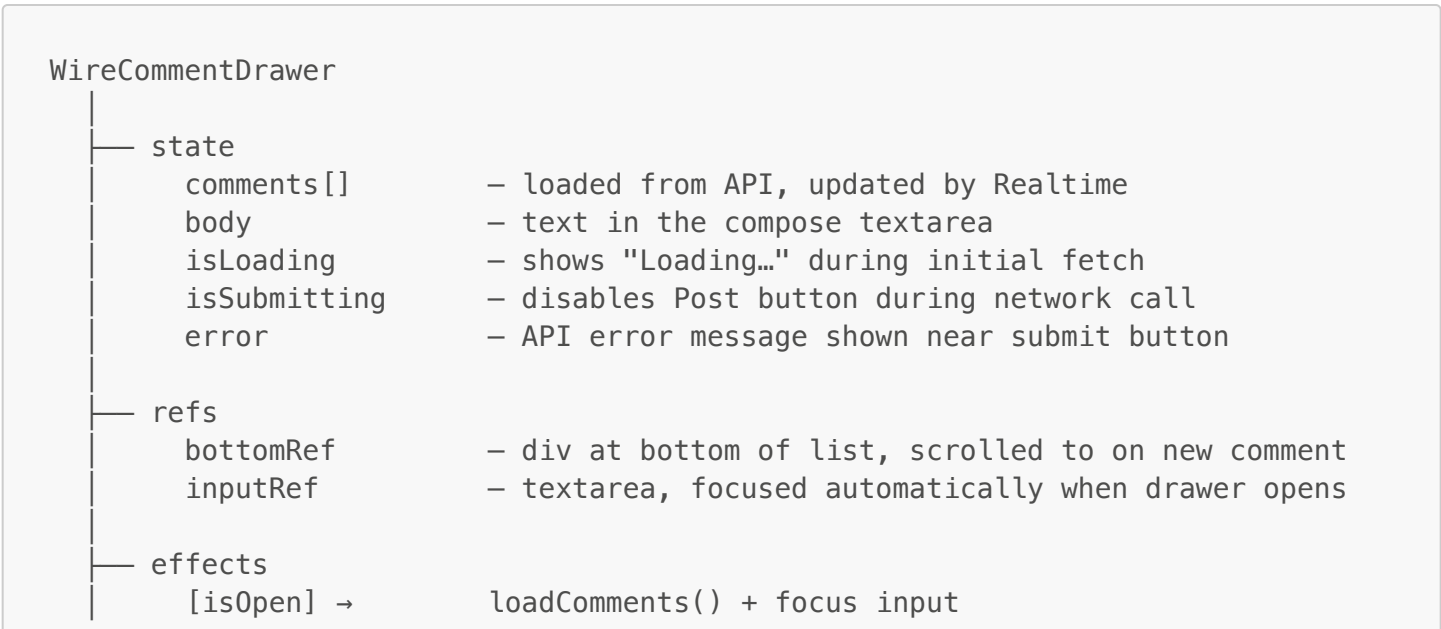
File: `frontend/components/WireCommentDrawer.tsx`

This is the largest and most complex piece of the feature. It is a fully self-contained React client component that manages: loading comments, live Realtime subscription, posting, deleting, and the full drawer UI.

Props Interface

```
interface Props {
  articleUrl: string; // URL of the article – used as the Supabase filter
  articleTitle: string; // Displayed in the drawer header
  isOpen: boolean; // Controls visibility
  onClose: () => void; // Called when the user closes the drawer
  currentUserId?: string; // Undefined if not logged in
  onCommentPosted?: () => void; // Called after a successful post (parent updates badge count)
}
```

Component Architecture



- | [isOpen, url] → Supabase Realtime channel subscribe/unsubscribe
- | [comments.length] → scroll to bottom
- └─ render
 - Backdrop (mobile only) → onPointerDown closes drawer
 - Drawer panel
 - Header (title + close button)
 - Scrollable comments list
 - Compose area (form or sign-in prompt)

Supabase Realtime — How It Works Step by Step

```
const channel = supabase
  .channel(`wire:${articleUrl}`)
  .on(
    "postgres_changes",
    {
      event: "INSERT",
      schema: "public",
      table: "wire_comments",
      filter: `article_url=eq.${articleUrl}`,
    },
    async (_payload) => {
      // Re-fetch the full list so profile joins are resolved
      const res = await fetch(`/api/wire/comments?url=${encodeURIComponent(articleUrl)}`);
      const data = await res.json();
      setComments(data.comments ?? []);
    }
  )
  .on(
    "postgres_changes",
    { event: "DELETE", schema: "public", table: "wire_comments" },
    (payload) => {
      setComments(prev => prev.filter(c => c.id !== payload.old.id));
    }
  )
  .subscribe();
```

Step by step:

1. `supabase.channel("wire:https://example.com/article")` — creates a named WebSocket channel. The name is arbitrary but unique per article URL, which ensures two drawers open on the same article share the same channel (Supabase deduplicates by name).
2. `.on("postgres_changes", { event: "INSERT", filter: ... })` — subscribes to INSERT events on the `wire_comments` table, filtered to rows where `article_url` matches. Without the filter, this handler would fire for every comment posted anywhere on the platform.
3. On INSERT: rather than using the raw `payload.new` (which has no profile join), the handler re-fetches the full comment list from the API. This ensures profile data (author name) is always available. The slight extra network round-trip is acceptable because new comments arrive infrequently and correctness matters more than speed here.
4. `.on("postgres_changes", { event: "DELETE" })` — note there is **no URL filter** on the DELETE handler. Supabase Realtime does not support filtered DELETE subscriptions (because the deleted row is already gone from the database by the time the event fires, so the filter can't be evaluated). Instead, the handler filters in JavaScript: `prev.filter(c => c.id !== payload.old.id)`. For a DELETE, `payload.old` contains the deleted row's data.

5. `return () => { supabase.removeChannel(channel); }` — the React effect cleanup function.

When the drawer closes (or the component unmounts), the WebSocket subscription is torn down. Without this, channels would accumulate and the browser would maintain open connections to articles the user is no longer viewing.

Optimistic UI on Post

```
// After successful POST:
setComments(prev => [...prev, data.comment]); // add to local list immediately
setBody(''); // clear the textarea
onCommentPosted?(); // notify parent to increment badge
```

"Optimistic UI" means the UI updates immediately when the user clicks Post, without waiting to see if the Realtime event arrives. The comment appears in the list the instant the API call returns `201`. This gives a snappy feeling even on slow connections.

A moment later, the Realtime INSERT event will also fire (because our own post triggers it). When it does, the handler re-fetches the full comment list — this results in a duplicate briefly. However, because comments are keyed by `c.id` (not index), React correctly reconciles the list and the duplicate is merged without a visible flash.

Avatar Component

```
function Avatar({ name }: { name: string | null }) {
  const initials = (name ?? "?").split(" ").map(n => n[0]).join("").slice(0, 2).toUpperCase();
  return (
    <div className="w-7 h-7 rounded-full bg-indigo-100 ...">
      {initials}
    </div>
  );
}
```

Generates initials from the user's full name ("Jane Smith" → "JS", "Admin" → "AD"). Falls back to "?" if the profile has no name. Profile photo (`avatar_url`) is fetched from the database but not yet displayed — it's reserved for a future iteration where profile images are uploaded.

The Backdrop

```
{/* Backdrop — only visible on mobile */}
<div
  className="fixed inset-0 bg-slate-900/40 z-40 lg:hidden"
  onPointerDown={onClose}
  aria-hidden="true"
/>
```

On desktop, the drawer overlays the right side of the screen without blocking the rest of the page. On mobile (screens narrower than `lg` breakpoint = 1024px), the drawer takes the full screen width (`w-full`) and the backdrop covers the rest. Tapping the backdrop closes the drawer. `onPointerDown` is used for the same reason as the sidebar overlay (see amendment v4.5.0 §2): it fires on first touch contact with no delay.

`aria-hidden="true"` hides the backdrop from screen readers — it is a decorative element with no meaningful content.

7. WireFeed Updates

Server Component Change — the-wire/page.tsx

The page server component was updated to pass the current user's ID down to the feed:

```
// Before
export default async function TheWirePage() {
  const { items, fetched_at } = await getWireItems();
  // ...
  return <WireFeed initialItems={items} fetchedAt={fetched_at} />;
}

// After
export default async function TheWirePage() {
  const [{ items, fetched_at }, user] = await Promise.all([
    getWireItems(),
    getUser(), // runs in parallel with the wire fetch
  ]);
  // ...
  return <WireFeed initialItems={items} fetchedAt={fetched_at} currentUserId=
{user?.id} />;
}
```

`Promise.all` runs both data fetches in parallel — the total page load time is the slower of the two (usually `getWireItems()`) rather than the sum of both. `user?.id` is `undefined` when not logged in, which the feed and drawer both handle gracefully.

Client Component Change — WireFeed.tsx

Three changes were made:

1. Comment count loading on mount:

```
useEffect(() => {
  fetch("/api/wire/comments/counts")
    .then(r => r.ok ? r.json() : null)
    .then(data => { if (data?.counts) setCommentCounts(data.counts); })
    .catch(() => {});
}, []);
```

This fires once when the feed renders. It's non-blocking — if it fails, `commentCounts` stays as an empty object and all articles show "Discuss". The effect has no dependencies (`[]`) so it only runs on first mount, not on every re-render or filter change.

2. Optimistic count increment:

```
function handleCountIncrement(url: string) {
  setCommentCounts(prev => ({ ...prev, [url]: (prev[url] ?? 0) + 1 }));
}
```

When a user successfully posts a comment, the drawer calls `onCommentPosted()`, which calls this function. The badge count on that article increases immediately in the feed behind the open drawer — so when the user closes the drawer, the badge already reflects their post.

3. Per-article discuss button and drawer trigger:

```
<button
  onClick={() => setDrawerArticle(item)}
  className="flex items-center gap-1 text-[11px] text-slate-400 hover:text-
```

```
indigo-600 ..."
>
  <ChatBubbleLeftIcon className="w-3.5 h-3.5" />
  {count > 0 ? count : "Discuss"}
</button>
```

`setDrawerArticle(item)` stores the entire `WireItem` object as the active article. The drawer renders conditionally:

```
{drawerArticle && (
  <WireCommentDrawer
    articleUrl={drawerArticle.url}
    articleTitle={drawerArticle.title}
    isOpen={true}
    currentUserId={currentUserId}
    onClose={() => setDrawerArticle(null)}
    onCommentPosted={() => handleCountIncrement(drawerArticle.url)}
  />
)}
```

Setting `drawerArticle` to `null` on close both hides the drawer and triggers the drawer's cleanup effect (which closes the Realtime WebSocket subscription).

8. Deployment Checklist

This section replaces the scary one-liner at the end of a coding session with a clear, step-by-step process you can follow yourself.

Step 1 — Run the Database Migration

This is the only step that cannot be reversed easily, so do it carefully.

1. Open the Supabase Dashboard for your project
2. Go to **SQL Editor** → **New query**
3. Copy the full contents of `supabase/migrations/025_wire_comments.sql`
4. Paste and click **Run**
5. Verify: go to **Table Editor** — you should see `wire_comments` and `wire_comment_upvotes` in the table list
6. Verify: go to **Database** → **Views** — you should see `wire_comment_counts`

Step 2 — Enable Realtime for wire_comments

Live comment updates will not work until this is done. Comment posting and reading will still work — only the real-time update will be missing.

1. In Supabase Dashboard, go to **Database** → **Replication**
2. Find the `wire_comments` table in the list
3. Toggle it **on** (the toggle should turn green)

That's it. No code changes needed — the migration's `ALTER PUBLICATION` line registers the table; toggling the Replication setting in the UI activates the WebSocket delivery.

Step 3 — Deploy the Application

If you deploy via Vercel (or similar):

```
git add .
git commit -m "Add Wire discussion threads"
git push
```

Vercel will automatically pick up the push and deploy. The new API routes under `/api/wire/comments/` and the `WireCommentDrawer` component will be live.

Step 4 — Verify in Production

1. Open `/the-wire` while logged in
2. Click "Discuss" on any article
3. The drawer should open with "No comments yet"
4. Type a comment and click Post
5. The comment should appear immediately
6. Open the same article in a second browser tab (or incognito) and open the drawer — your comment should be visible
7. Post another comment in the second tab — it should appear in the first tab's drawer within 1–2 seconds (Realtime working)
8. Click "delete" on your own comment — it should disappear
9. Close the drawer — the article's "Discuss" button should now show "1" (the comment you posted, minus the one you deleted)

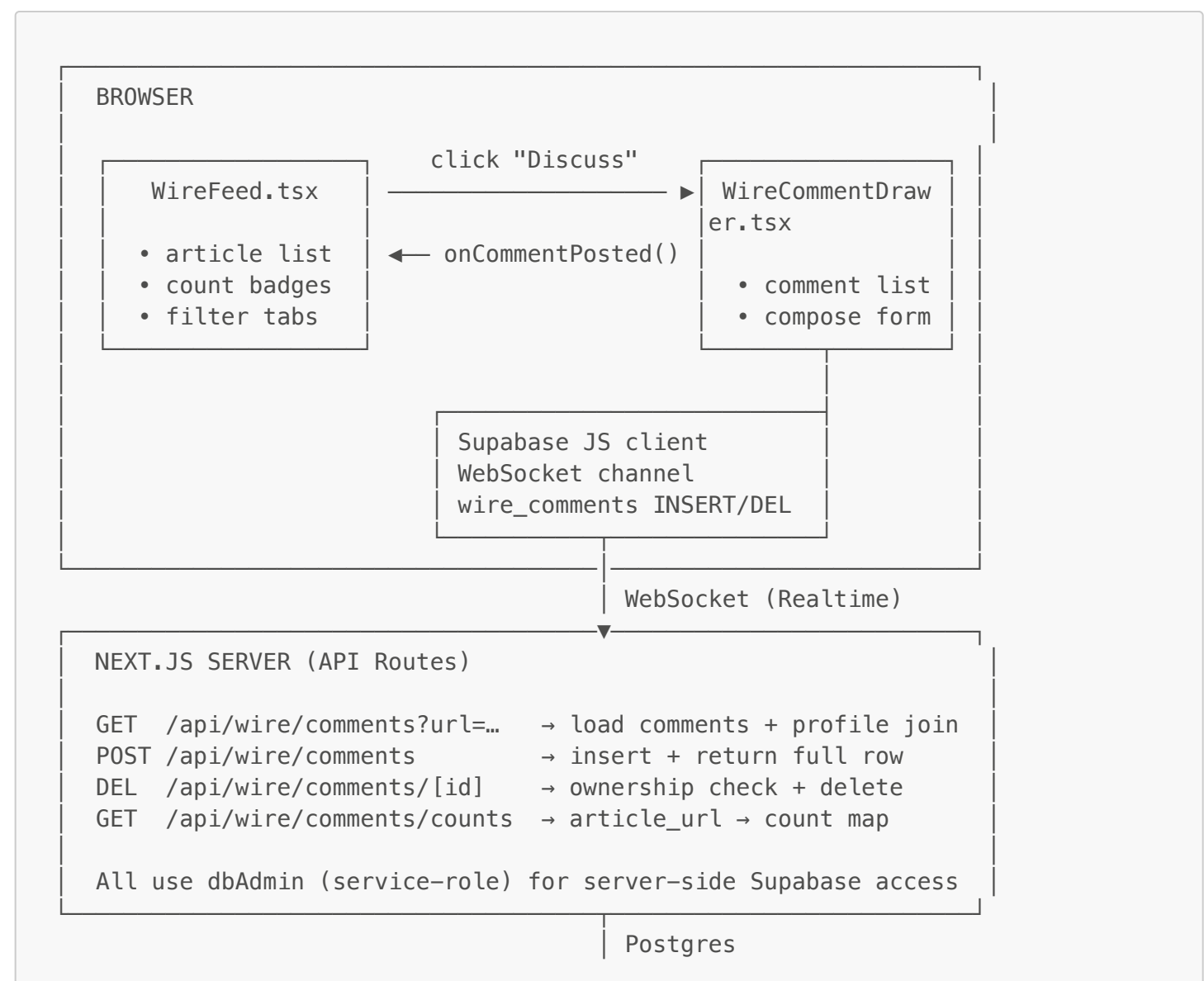
What Happens If Realtime Is Not Enabled?

The feature degrades gracefully:

- Initial comment load: works normally
- Posting a comment: works normally (appears via optimistic UI)
- Seeing other users' new comments without refreshing: does **not** work
- Deleting a comment: works normally

No errors appear. The drawer simply doesn't update unless the user closes and reopens it.

9. Architecture Reference Diagram



▼

```
SUPABASE DATABASE

wire_comments      (data table, RLS enabled, Realtime on)
wire_comment_upvotes (junction table, trigger-managed count)
wire_comment_counts (view: GROUP BY article_url)

Policies:
  SELECT → everyone
  INSERT → auth users only, user_id = auth.uid()
  DELETE → own rows only
```

10. Known Issues Register

Issue	Status	Amendment
Right sidebar AI input buried on desktop	Resolved	v4.5.0
Mobile sidebar requires two taps to close	Resolved	v4.5.0
AI response quality not measurable	Resolved	v4.5.0
The Wire RSS instability (Google News)	Resolved	v4.5.0
Hospital quality scores missing from dashboard	Resolved	v4.5.0
Payment failure has no recovery UI	Resolved	v4.5.0
State comparison view missing	Resolved (was already built)	v4.6.0
Wire articles have no discussion capability	Resolved	v4.6.0
Wire comment upvote button (UI only — DB ready)	Open	—
Weekly analytics digest email	Deferred (email provider not configured)	—